Aran: UI Design Spec

For: Inbathamizhan S (designing the new UI in Claude Design) and the team. **Read first:** Learn 15: UI Design Specs and Design Tokens.

This spec lists every page of the Aran console, and for each page every section, every piece of content, every component, the data it shows and the exact API field it comes from, what the user can do, what moves, and how it looks when it is loading, empty, failing or disabled. It describes what the current site does (`app/web`, served by the FastAPI server in `app/server`) so a new design can replace the look without losing any behaviour.

No colours are given. Every colour is a named placeholder such as `{primary-color}` or `{input-color}`. Pick the values yourself in Claude Design; section 1 lists every placeholder and what it is for.

Contents

0. How to read this spec
1. Design tokens (placeholders to fill in)
2. App shell (frame around every page)
3. Component library
4. Fixed vocabularies (data types, systems, purposes, samples, drills, roles)
5. Page 1: Guided tour (13 chapters)
6. Page 2: Overview
7. Page 3: Live traffic
8. Page 4: Customers and erasure
9. Page 5: Leak trace
10. Page 6: Immune layer
11. Page 7: Results and tests
12. Page 8: How it works
13. API reference (every endpoint the UI uses)
14. Responsive and accessibility rules
15. Design checklist before handoff
16. Optional extra pages (need new endpoints)

---

0. How to read this spec

Notation	Meaning
`{name-color}`	Colour token. You choose the value, once for light and once for dark theme.
`{space-3}`, `{radius-card}`, `{motion-medium}`	Other tokens (spacing, shape, motion).
`GET /api/stats → requests`	Binding: the page calls this endpoint and shows this field of the JSON reply.

Notation	Meaning
`poll 3 s`	The page re-calls the endpoint every 3 seconds while open.
`once`	Called once when the page or chapter opens.
Copy: "…"	Suggested text. Keep the meaning; wording can change.
States	Loading / Empty / Error / Disabled / Success unless stated otherwise.
`[needs endpoint]`	Not supported by the server yet.

Global data rules (these are part of Aran's pitch, keep them visible in the design):

- **Every number is live.** Nothing on any page is typed into the design except the Problem chapter's breach facts and the team names.
- **Identifiers are generated.** Every Aadhaar, card, PAN, phone and UPI ID on screen is checksum-valid but belongs to no one. The top bar says so on every page.
- **Three meaning colours.** `{real-data-color}` = a real customer value, `{token-color}` = an Aran token, `{leak-color}` = something that got out raw, an alert or a failed check. Use them for nothing else, and always pair them with an icon or word.

---

1. Design tokens (placeholders to fill in)

1.1 Colour

Token	Used for
`{background-color}`	Page background behind everything
`{surface-color}`	Cards, sidebar, top bar
`{surface-2-color}`	Inset areas inside cards: log-line boxes, table header row, code blocks, skeletons
`{border-color}`	Card borders, table row lines, dividers, input borders at rest
`{text-color}`	Main text, headings, stat values
`{text-2-color}`	Secondary text: notes, captions, "muted" small text, table meta
`{brand-color}`	Logo text "Aran", the eyebrow labels above page titles
`{brand-fill-color}`	Logo mark square background
`{primary-color}`	Primary buttons, active nav item, active tab, active chip, links, focus of the current tour chapter
`{on-primary-color}`	Text and icons on top of `{primary-color}`
`{accent-color}`	Highlights inside text ("Most likely source: **loan-vendor**"), the Verhoeff dial needle, running totals, logo mark shape
`{input-color}`	Background of text inputs, textareas, selects, number fields

Token	Used for
`{input-border-color}`	Input border at rest
`{input-focus-color}`	Input border when focused
`{placeholder-color}`	Placeholder text in inputs
`{focus-ring-color}`	Keyboard focus outline on every focusable element
`{real-data-color}` / `{real-data-bg-color}`	Raw sensitive values highlighted in “Before” text; real-identifier digit strips; raw packets in the pipeline
`{token-color}` / `{token-bg-color}`	Aran tokens highlighted in “After” text; token digit strips; token packets; AI placeholders like `<AADHAAR_1>`
`{leak-color}` / `{leak-bg-color}`	Alerts, “passed raw” pills, failed checks, errors, Erase buttons, destroyed keys, the leaked node on the trace map
`{success-color}` / `{success-bg-color}`	“✓ valid”, “alive” key pills, passed tests, “caught” disguise cards, the trace verdict when correct
`{warning-color}`	Retention lock (🔒 KYC locked 5 years), “learning” mode, monitor-only pill
`{info-color}`	Neutral info notes, “checking…” packet, the scan ring on the firewall shield
`{clean-color}`	Packets that carried nothing sensitive (“passed clean”)
`{band-color}`	The learned “normal” band on the immune radar
`{chart-grid-color}`	Grid lines and axes on radar, gauges and bars
`{bar-track-color}` / `{bar-fill-color}`	Track and fill of confidence bars and rank bars
`{threshold-marker-color}`	The threshold tick on confidence bars and on the gauge
`{overlay-color}`	Mobile menu backdrop, confirm dialog backdrop
`{shadow-color}`	Card and popover shadows

Every text-on-background pair above must reach 4.5:1 contrast in both themes (3:1 for large numbers over 24 px).

1.2 Type

Token	Role
`{font-display}`	Page titles (h1), chapter headlines
`{font-body}`	All other text
`{font-mono}`	Every identifier, token, number, hash, log line, latency, code
`{font-tamil}`	“அரண்” next to the logo and under the shield (must support Tamil script)
`{text-size-h1}`, `{text-size-h2}`, `{text-size-h3}`	Page title, section title, card title
`{text-size-body}`, `{text-size-small}`, `{text-size-caption}`	Body, notes, eyebrow and axis labels

Token	Role
`{text-size-stat}`	Big numbers in stat tiles and count-ups
`{weight-regular}`, `{weight-medium}`, `{weight-bold}`	

1.3 Space, shape, depth, motion

Token	Role
`{space-1}` ... `{space-8}`	Spacing scale (e.g. 4, 8, 12, 16, 24, 32, 48, 64 px; your choice)
`{radius-card}`, `{radius-input}`, `{radius-pill}`, `{radius-cell}`	Corners of cards, inputs/buttons, pills/chips, digit cells
`{border-width}`	Card and input borders
`{shadow-card}`, `{shadow-popover}`	Resting card; dialogs, menus, tooltips
`{motion-fast}`	Hover, press, focus (about 120 ms)
`{motion-medium}`	Cards appearing, bars filling, tab switches (about 300 ms)
`{motion-slow}`	Count-ups, gauge needle, scene transitions (about 1 s)
`{motion-step}`	Time per step in step-by-step explainers (330 to 900 ms; each scene states its own)
`{ease-standard}`, `{ease-emphasised}`	Normal easing; the packet and beam curves

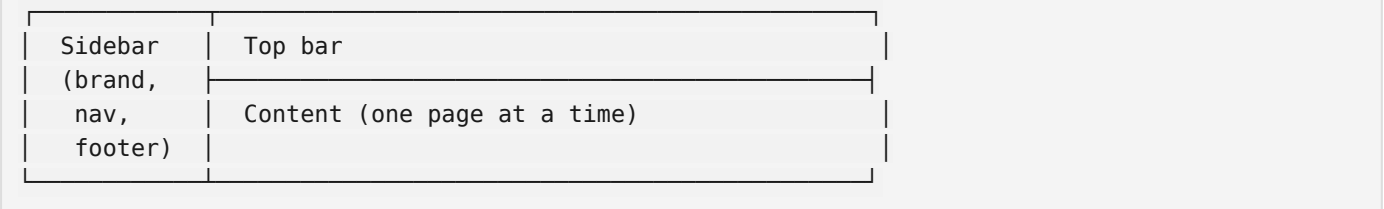
1.4 Themes

Two themes, **dark (default)** and **light**, switched by the top-bar button and remembered on the device (browser storage, key `aran-theme`). Every colour token above has one value per theme. Nothing else changes between themes.

---

2. App shell (frame around every page)

2.1 Layout



2.2 Sidebar

Part	Content	Behaviour
Brand	Logo mark (a fort battlement shape in `{accent-color}` on a `{brand-fill-color}` square, with a small `{token-color}` block in the lower right) + "**Aran**" + "அரண்" in `{font-tamil}`	Not clickable (or links to the tour)
Nav	8 items in this order: Guided tour · Overview · Live traffic · Customers & erasure · Leak trace · Immune layer · Results & tests · How it works	Current page uses `{primary-color}`. Clicking changes the URL hash (`#tour`, `#overview`, `#traffic`, `#customers`, `#trace`, `#immune`, `#results`, `#about`) so pages can be bookmarked and the browser Back button works.
Footer	"Source on GitHub" (opens https://github.com/itzz-inbathamizhans/aran in a new tab) · "KARE · Banking & FinTech"	

Optional addition: a small live badge next to "Immune layer" showing `GET /api/stats → open_alerts` when above 0, in `{leak-color}`.

2.3 Top bar

Part	Content	Behaviour
Menu button (phone only)	☰	Opens the sidebar as a drawer over `{overlay-color}`; tapping a nav item or the backdrop closes it
Environment strip	**Copy:** "LIVE · every value computed by Aran's code · identifiers are generated, never real people"	Always visible; on phone it may shorten to "LIVE · generated identifiers only"
Theme toggle	"☀ Light" when dark, "☾ Dark" when light	Switches theme instantly; remembered

2.4 Whole-app states

State	Trigger	Show
Starting	`GET /api/health → ready` is `false` (poll every 2 s until true)	Full content-area card: spinner, "Starting Aran", **Copy:** "Loading the trained detector, NER and context models, and enrolling customers. This takes a few seconds after a cold start." No page content renders until ready.
Load failed	`GET /api/health → error` is set	Same card, text "Load failed: {error}" in `{leak-color}`

State	Trigger	Show
Server unreachable	The health call itself fails	Keep the Starting card and keep retrying every 2 s
Any endpoint 503	A page call returns 503	Inline error note with the server message (“Aran is loading its models, try again in a few seconds”)

2.5 Page header pattern (every console page)

1. **Eyebrow:** small caps label in `{brand-color}` (given per page).
2. **Title (h1):** one sentence that states the idea.
3. **Lede:** one or two lines in `{text-2-color}`.

---

3. Component library

Design these once, on their own board, with every state.

#	Component	Anatomy	Variants and states
C1	**Button**	Label, optional leading icon	Primary (`{primary-color}`), Secondary (outline), Ghost (text only), Danger (`{leak-color}`). States: default, hover, focus, pressed, disabled, busy (label changes, e.g. “Scanning…”, “Running…”)
C2	**Card**	Container on `{surface-color}` with `{border-color}`, `{radius-card}`, `{shadow-card}`; optional title (h3) and a right-aligned action	Default; tone “danger” (left edge or border in `{leak-color}`)
C3	**Stat tile**	Big value in `{font-mono}` `{text-size-stat}`, label below, optional note in `{text-2-color}`	Default, danger tone (value in `{leak-color}`), no data (“–”), loading (skeleton)
C4	**Eyebrow**	Uppercase small label	—
C5	**Pill**	Short status word on a tinted background	ok (`{success-color}`), danger (`{leak-color}`), warning (`{warning-color}`), neutral

#	Component	Anatomy	Variants and states
C6	**Chip**	Rounded selectable label	Static, selectable (off / on in `{primary-color}`), result chips ok (`{success-*}`) and bad (`{leak-*}`), mono chips for tokens
C7	**Tabs**	Two or more tab buttons above a panel	off, on, focus
C8	**Segmented radio**	Two or more labelled radio options in a row	selected, unselected, focus
C9	**Text input / textarea / select / number**	Field on `{input-color}`, `{input-border-color}`, label, placeholder in `{placeholder-color}`	default, focus (`{input-focus-color}`), disabled, error; mono variant for numbers and logs
C10	**Table**	Header row on `{surface-2-color}`, rows divided by `{border-color}`; numeric columns right-aligned in `{font-mono}`	Default, clickable rows (hover highlight, pointer), small (dense), horizontal scroll inside the card on phone
C11	**Log line block**	Pre-formatted text in `{font-mono}` on `{surface-2-color}`, wraps long lines	Plain; with marks (C12); scrollable output (max height, used for test output)
C12	**Marked text**	Log line where each finding is highlighted. Raw value mark: `{real-data-bg-color}`; tooltip “Aadhaar · 98%”	mark default, mark active (outline), mark clickable
C13	**Token text**	Log line where each issued token, `<OTP:destroyed>` and `<TYPE:withheld-erased>` is highlighted in `{token-bg-color}`	—
C14	**ID strip**	One cell per digit in `{font-mono}`, last cell (check digit) emphasised, caption with icon: “✓ passes Verhoeff · real (generated customer)” or “⊘ fails Verhoeff · protected Aran token”	real (`{real-data-*}`), token (`{token-*}`), unknown (!)
C15	**Confidence bar**	Track, fill to probability `p`, threshold tick	Fill `{bar-fill-color}`; tick `{threshold-marker-color}`; aria “confidence 98%”
C16	**Gauge**	Half-circle, needle at `p`, threshold tick, centre text “98%”	Needle animates `{motion-slow}`

#	Component	Anatomy	Variants and states
C17	**Rank bar row**	Label, bar with fill = share, value “12 · 80%”	—
C18	**Alert box**	Title “src → sink”, time and score, bullet reasons, mono feature line, actions	open (actions visible), confirmed attack (pill danger), marked normal (pill ok); pop-in animation
C19	**Inline notes**	One line under a form	ok (`{success-color}` “✓ …”), error (`{leak-color}`), info
C20	**Skeleton**	Grey block in `{surface-2-color}` the shape of the content, with a slow shimmer	Used while a tour chapter’s first request runs
C21	**Spinner**	Small rotating ring	Starting screen, busy buttons (optional)
C22	**“Under the hood” disclosure**	Collapsed by default, summary “Under the hood”, opens to small text with theory, measured result and code path	closed, open
C23	**Step controls**	▶ Play / ❚❚ Pause / ▶ Continue / ↺ Replay, ‹ back, › forward, “Skip to end”, counter “7/15”	Shared by every step-by-step scene
C24	**Count-up number**	Number that counts from 0 to its value	Duration about 1.1 s (`{motion-slow}`), ease-out; reduced motion: show final value
C25	**Verdict banner**	One line, icon + bold phrase	ok (`{success-*}`), bad (`{leak-*}`)
C26	**Receipt card**	Title, key-value table of the erasure receipt, actions	See page 4
C27	**Merkle chain**	Row of hash boxes “erasure → level 1 → … → root”, each with first 10 hex chars	Boxes light up one by one (380 ms each)
C28	**Key icon**	Key drawing	alive, locked (with 🔒), gone (shatters into 5 shards that fly apart), ghost (outline, no customer yet)
C29	**Radar chart**	7 axes, learned band, grid rings, window polygons	Window normal, window alert (`{leak-color}`), newest window emphasised

#	Component	Anatomy	Variants and states
C30	**Chat bubble**	“who” label + text; placeholders highlighted as tokens	staff (one side), provider (other side); fade-in when its step is reached
C31	**Confirm dialog**	Title, consequence sentence, Cancel, Danger confirm	Used before erasing a key (currently the browser’s own confirm)
C32	**Member card**	Initials avatar, name, role	Team section
C33	**Tooltip**	Short label on hover/focus	On marks, nav dots, threshold ticks

---

4. Fixed vocabularies

Use these exact labels (from the server and `components.jsx`).

Data types (field `type` in findings): `AADHAAR` Aadhaar · `CARD` Card · `MOBILE` Mobile · `PAN` PAN · `OTP` OTP · `UPI` UPI ID · `PER` Name · `ADDR` Address.

Sinks (systems that store data; `GET /api/config → sinks[]` with `name`, `purpose`, `threshold`):

Sink	Display name	Purpose	Threshold
`log-platform`	Log platform	Service	0.5
`crm-notes`	CRM notes	Service	0.5
`loan-vendor`	Loan vendor	Loans	0.5
`llm-gateway`	AI chatbot (AI gateway)	Service	0.3
`marketing-cloud`	Marketing	Marketing	0.5
`kyc-archive`	KYC archive	KYC	0.5

Free-text sinks are allowed on the process endpoint (lower-case letters, digits, dots, dashes), e.g. `paste-site.example` from the drill.

Sources (systems that send): `console` (You), `loan-origination` (Loan app), `onboarding`, `call-centre`, `upi-switch`, `mobile-app`, `employee-desktop`, `core-banking`, `guided-tour`.

Purposes (one key per customer per purpose): KYC · Loans · Service · Marketing. KYC is retention-locked for 5 years (PMLA).

Samples (`GET /api/config → samples`): `loan` Loan application API · `kyc` KYC upload API · `callnote` Call-centre note · `payment` Payment callback · `otp` Login with OTP · `chatbot` Prompt to an AI assistant. Each has a source and sink in `sample_sources` / `sample_sinks`.

Drills (`GET /api/config → drills`): `new-sink` Send customer records to a sink never seen before · `bulk-export` Bulk export of loan applications in one window · `detok-spike` Insider detokenises many customer tokens · `bypass` Firewall switched to monitor-only on the log platform.

AI gateway roles: `customer-care` Customer care · `kyc-officer` KYC officer · `analyst` Analyst.

Evidence labels (keys of `finding.evidence`): checksum “Checksum passes (Verhoeff / Luhn)” · kw “Type word nearby” · iin “Card issuer prefix” · len16 “16 digits” · plus91 “+91 prefix” · ctx “Context model score” · otp_ctx “OTP context model” · upi_ctx “UPI context model” · registry “Handle in UPI registry” · handle “UPI handle” · hdot “Dot in handle” · ner “AI name/address model” · ntok “Span length” · digit “Contains digits” · calendar “Looks like a date” (counts against) · idword “Looks like an order ID” (counts against) · title “Title case” · deva “Devanagari script” · cue “Cue word before” · pin “PIN code inside” · comma “Comma separated” · keyfield “Phone-like field name” · sendto “‘called from’ / ‘sent to’”.

Immune features (radar axes, in order of `names`): volume_z volume · raw_frac_z left raw · hour_distance odd hour · pair_novel new route · class_novel new data type · detok_log read-backs · cust_ratio customers.

4.1 The finding object (used on many pages)

Every protected request returns an **event**:

```
id, at (unix seconds), source, sink, before (text sent), after (text stored),
latency_ms, bypass (true if monitor-only), trace_id,
findings[]: type, start, end (character span in before), p (0–1),
            protected (true/false), token (what replaced it, if any),
            evidence {name: number}, registry ("hit" or null), repaired (true/false)
```

---

5. Page 1: Guided tour

Route: `#tour`, chapters `#tour/0` … `#tour/12`. **Default landing page. Purpose:** explain Aran to judges and newcomers in 13 short chapters, each a moving picture of a real server response, with only a headline and one line of text.

5.1 Tour frame (around every chapter)

Part	Content	Behaviour
Chapter dots (desktop)	13 numbered dots with the chapter name on hover/focus: 1 Start · 2 The problem · 3 Simple filters fail · 4 Check digits · 5 Building a token · 6 Finding data · 7 Disguises · 8 Tracing a leak · 9 Erasure · 10 Immune system · 11 AI chatbots · 12 Evidence · 13 Glossary	States: current (`{primary-color}`), seen (filled softly), not seen. Click jumps.
Chapter select (phone)	Dropdown “1. Start” … “13. Glossary”	Replaces the dots under 720 px
Chapter header	Headline (h1, `{font-display}`) + one tagline	Given per chapter below
Chapter body	The scene	Fades/slides in on each chapter change (`{motion-medium}`)
“Under the hood”	Disclosure (C22) at the bottom of most chapters	Closed by default

Part	Content	Behaviour
Bottom nav	Left: “← {previous chapter name}” (disabled on chapter 1). Middle: “← → keys work too”. Right: “{next chapter name} →”; on the last chapter “Open the full console →” (goes to Live traffic)	Left/Right arrow keys move chapters unless the cursor is in an input. Scroll to top on change.

Each chapter that fetches data shows a **skeleton** (C20) until its first reply, an **error note** under the scene on failure, and a **refresh button** (“↻ …”) that is disabled with a busy label while running.

5.2 Chapter 1: Start

- **Headline:** “A firewall for data”. **Tagline:** “Aran swaps customer numbers for safe tokens before any system stores them. Watch real requests flow through it.”
- **Scene: firewall pipeline** (wide SVG, about 900 × 400 units; scrolls inside its frame on phone).
  - Left column, 7 source boxes, caption “BANK SYSTEMS SEND”: You · Loan app · Onboarding · Call centre · UPI switch · Mobile app · Staff desktop.
  - Centre: a shield labelled “ARAN” and “அரண்”, two rings around it.
  - Right column, 6 sink boxes, caption “SYSTEMS THAT STORE IT”: Log platform · CRM notes · Loan vendor · AI chatbot · Marketing · KYC archive.
  - Curved lanes from every source to the shield and from the shield to every sink.
- **Data:** `POST /api/tour/packet` (body `{}`) returns one event (4.1) plus `kind`. Own text uses `POST /api/process {text, source: "console", sink}`.
- **Animation (per packet, total 3.35 s):**
  1. A pill-shaped packet in `{real-data-color}` labelled with up to 3 data types found (“Aadhaar · Card · Mobile”, or “no secrets”) travels along the lane from its source to the shield (1.1 s, `{ease-emphasised}`). The source box and lane light up.
  2. At the shield the rings pulse in `{info-color}` and a pill “checking…” sits under it (0.45 s).
  3. A packet in `{token-color}` labelled “3 tokens” (or `{clean-color}` “passed clean”) travels to its sink (1.1 s). The sink lane lights.
  4. The sink gains a counter badge (+1). Packet disappears (0.7 s rest).
  - Several packets may be in flight at once.
- **Auto-stream:** on open, sends one packet, then one every 2.6 s, 14 times, then stops.
- **Legend row:** swatch real customer data · swatch Aran tokens · swatch nothing sensitive · right side mono counter “{n} requests · {n} values protected”.
- **Controls:** “Send a real request” (primary) · “▶ Stream requests” / “❙❙ Stop the stream” (secondary).
- **Own message row:** mono input, placeholder “Or type your own message, e.g. my card is 4111 1111 1111 1111, OTP 482913” · select “to Log platform / to CRM notes / …” (default AI chatbot) · “Send” (disabled when empty).
- **Before/after panel** (latest packet): left header “Sent by {source}” with Marked text (C12); arrow “→”; right header “Stored by {sink} · {latency_ms} ms” with Token text (C13). Re-animates on each new packet.
- **Three pillars** below (stagger in 120 ms apart): 🔍 **Find** “even hidden or misspelt values” · 🛡 **Protect** “tokens that can never pass as real” · 🧾 **Prove** “trace leaks, erase with a receipt”.

- **Button:** “Start the tour →”.
- **Reduced motion:** packets jump to the sink; badge updates; no ring pulse.
- **Accessible description:** “Requests flow from bank systems through Aran to the systems that store them.”

5.3 Chapter 2: The problem

- **Headline:** “Leaks come through side doors”. **Tagline:** “Not the main database: logs, vendors, tickets and AI chatbots.”
- **Three stat tiles (danger tone), count up:** “₹40.9 cr · average cost of one breach, Indian finance · IBM, 2026” · “₹250 cr · maximum DPDP Act penalty · DPDP Act, 2023” · “35 M · masked card records sold from one firm · Juspay, 2020”. (Static facts; the only hard-coded numbers in the app.)
- **Breach timeline:** a horizontal line with 5 dots (2016 3.2 M debit cards · 2019 SBI SMS logs · 2020 35 M masked cards · 2023 Code in ChatGPT · 2024 31 M Star Health). Clicking a dot opens a card below: what happened (e.g. “Records sold through Telegram bots”) + a danger pill with the lesson (e.g. “Who leaked it?”). Default: last dot. Dots appear one by one (120 ms stagger).
  - 2016: Malware at a payment processor · Leaks start at partners
  - 2019: Server logs left open, balances visible · Log files leak
  - 2020: Juspay records sold months later · First sign is a dump outside
  - 2023: Samsung staff pasted secrets into AI · AI is a new exit
  - 2024: Records sold through Telegram bots · “Who leaked it?”
- **Four question cards** (stagger 150 ms), each with a “?” badge: “Where is each customer’s data now?” · “Which copies are protected?” · “Who leaked this file?” · “Can we prove erasure?”
- **Closing line:** “Banks cannot answer these four today. Aran answers all four.”
- **Under the hood:** “Problem analysis, threat model and sources: `docs/research/aran-research.md` sections 1 and 3.2.”
- No API calls.

5.4 Chapter 3: Simple filters fail

- **Headline:** “Why simple search fails”. **Tagline:** “Every red square is a harmless number a plain filter would block. These are 300 real production log lines with no customer data at all.”
- **Data:** `POST /api/tour/false-alarms` (once, and on refresh) → `lines`, `systems`, `naive` {Aadhaar-shaped, Card-shaped, OTP-shaped}, `naive_total`, `aran` {type: count}, `aran_total`, `cells` [[naive, aran] per line], `examples[]` {line, naive[], aran[]}, `seconds`.
- **Scene: two grids side by side**, one cell per log line (300 cells, e.g. 20 × 15):
  - Left card “Plain pattern search”, count-up of `naive_total` (1.6 s), sub-line “false alarms in {lines} real log lines”. Cells with hits fill with `{leak-color}`, opacity scaled by hit count.
  - Right card “Aran”, count-up of `aran_total`. Cells with Aran hits fill the same way (almost all stay empty).
  - Cells appear one after another (5 ms stagger) like rain.
- **Button:** “↻ 300 new lines” (busy: “Scanning…”).
- **Under the hood:** table of up to 6 example lines: Real log line (mono) · Plain pattern saw (`{leak-color}`, e.g. “OTP-shaped: 50010”) · Aran (“nothing” or types). Text: “Lines from

Loghub (16 systems). On all 32,000 lines (E1): OTP pattern 67,481 false alarms vs 6 for Aran; cards 6,086 vs 81; Aadhaar 366 vs 4. Code: `research/experiment.py`."
- **States:** skeleton while loading; error note.

5.5 Chapter 4: Check digits

- **Headline:** "The last digit is a lock". **Tagline:** "Every Aadhaar and card ends in a check digit computed from the others. Real numbers always pass. Watch the maths."
- **Inputs row:** mono input (digits and spaces only, placeholder "12-digit Aadhaar or 16-digit card") · "Check" · "New Aadhaar" · "New card".
- **Tweak row** (only when 12 or 16 digits): "✎ Mistype one digit" (changes digit 5 by +3) · "⇄ Swap two neighbours" (swaps digits 3 and 4). Both re-run the check.
- **Data:** `GET /api/tour/generate` → `aadhaar`, `card` (on open, generates an Aadhaar). `POST /api/tour/checksum {digits}` → `kind` ("Aadhaar (Verhoeff)" or "Card (Luhn)"), `digits`, `valid`, `steps[]`, plus `final` (Verhoeff) or `total` (Luhn), `correct_check_digit`. Steps: Verhoeff {position, digit, permuted, before, after}; Luhn {position, digit, doubled, value}. Error 400: "enter 12 digits (Aadhaar) or 16 digits (card)".
- **Scene:**
  - **Digit row:** one cell per digit; the check digit cell emphasised. Steps run from the last digit to the first; the current cell is "now" (`{primary-color}` ring), done cells are "done". Under each done cell: the running value (Verhoeff) or the doubled value (Luhn, with "×2" tag on doubled positions).
  - **Verhoeff panel:** a **dial** (pentagon in the centre, 10 numbered circles around it, a needle) that rotates to the running value (36° per step), with value 0 marked as the goal. Text: "digit at position {n}", "{digit} shuffled to {permuted}", "{before} combined → **{after}**". Before play: "Press play to run the Verhoeff check from the last digit".
  - **Luhn panel:** a horizontal **sum bar** filling to the running total, with tick marks at every multiple of 10. Text: "digit at position {n}, doubled", "{digit} ×2 → {value} running total **{sum}**".
  - Step time 520 ms; auto-plays on each new check; Step controls (C23) label "Run the check".
- **Verdict (C25) when done:** valid: "✓ Ends at 0 (or {total}, a multiple of 10): **could be a real number**". Invalid: "✗ Ends at {final/total}: **cannot be real** · the last digit should be {correct_check_digit}".
- **Under the hood:** Verhoeff text (dihedral group D5, the dial, catches every single typo and neighbour swap) or Luhn text, with `research/aran_core.py`.
- **Reduced motion:** show all steps completed and the verdict.

5.6 Chapter 5: Building a token

- **Headline:** "Building a token". **Tagline:** "Same length, same look, but the last digit is wrong on purpose, so no scanner can ever mistake it for a real number."
- **Sink chips:** "for Log platform" · "for CRM notes" · "for Loan vendor" · "for Marketing" (one on). Changing the sink rebuilds the same Aadhaar for that sink.
- **Data:** `POST /api/tour/build-token {sink, aadhaar?}` → `aadhaar`, `body` (first 11), `check`, `sink`, `rounds[10]` {round, left, right, aes (hex)}, `encrypted`, `right_check`, `offset` (1–9), `token`, `token_valid` (false), `real_valid` (true), `firewall_token`, `firewall_after`, `matches`, `note`.
- **Scene: token forge** (15 frames, 700 ms each, auto-plays; Step controls label "Build the token"):

    - Frame 0: big digit cells of the real Aadhaar, check digit in `{success-color}`. Caption “A real (generated) Aadhaar: the last digit passes Verhoeff”.
    - Frame 1: check digit set aside (shown as “·”). “Keep the first 11 digits; set the check digit aside”.
    - Frames 2–11: the 11 digits split into two halves (two tints), digits roll to the new values of each round (25 ms stagger per digit). Caption “Encryption round {k} of 10: one half is mixed with an AES fingerprint of the other, then they swap”. Under it: mono “AES → {aes}… tweak”{sink}““. A row of 10 round dots fills.
    - Frame 12: ghost check digit = `right_check`. “The correct check digit for the new digits would be {right_check}”.
    - Frame 13: check digit becomes wrong in `{leak-color}`. “Add this system’s secret offset (+{offset}): {right_check} + {offset} → {bad}”.
    - Frame 14: whole token in `{token-color}`. “The token: same length, same look, guaranteed to fail Verhoeff”.
    - Result block: row “Real {aadhaar} ✓ passes” (ok pill) · row “Token {token} ✗ fails” (danger pill) · line “✓ The live firewall issued exactly this token when the Aadhaar was sent to {sink}” (or “✗ The live firewall issued {firewall_token}”).
- **Button:** “↻ Another Aadhaar”. **Note:** “Pick another system: the same Aadhaar becomes a different token for each one. That is what makes leaks traceable (chapter 8).”
- **Under the hood:** Theorem 1 (perfect separability), FF1 NIST SP 800-38G, tweak = system name, keyed offset, vault tokens for enrolled customers, E2 result (0 of 20,000), code paths.

5.7 Chapter 6: Finding data

- **Headline:** “Finding what is sensitive”. **Tagline:** “Names, phones, OTPs and UPI IDs have no check digit. Aran weighs clues and gives each value a measured probability.”
- **Sample chips:** Hinglish call note (default) · Loan application · Chatbot prompt · UPI payment log · OTP log · KYC record.
- **Data:** `POST /api/sample {kind}` → event (4.1).
- **Scene:**
    - **Scan box:** the request text as Marked text (C12); a scan line sweeps once over it when it arrives. Hint “Tap any highlighted value.”
    - **Finding chips:** one per finding, “Aadhaar 99%”, “Name 87%” … ; first selected by default.
    - **Clue panel** for the selected finding: left a **Gauge** (C16) at `p` with the threshold tick, caption “replaced at or above the mark”; right the type and value (mono), then one row per evidence entry: clue label + small bar (filled to the value, clamped 0–1; NER logit squashed) animating in one after another (90 ms stagger). Clues that count against (“looks like a date”, “looks like an order ID”) are styled as negative. If `registry` is set: row “bank’s own customer list ✓ match”.
    - **Stored panel:** header “What was stored · {latency_ms} ms”, Token text of `after`.
- **Under the hood:** logistic stacker + isotonic calibration (ECE 0.015, E7), NER F1 0.85/0.76/0.69 (E5), HMAC cuckoo-filter registries; code paths.

5.8 Chapter 7: Disguises

- **Headline:** “Disguises do not work”. **Tagline:** “One Aadhaar hidden 12 ways, each sent through the live firewall. Aran undoes the disguise, then checks.”
- **Data:** `POST /api/tour/evasion` → `customer`, `aadhaar`, `rows[12]` {trick, before, after, caught, latency_ms}, `caught`.

- **Scene:** big counter "{caught}/12 disguises caught" (counts as cards flip). Grid of 12 cards (4 × 3 desktop, 2 × 6 phone), each: trick name · disguised input (mono, "customer update" prefix removed). Cards flip one by one (330 ms each): the output appears (mono, token highlighted) with a badge ✓ (`{success-color}`, card "caught") or ✗ (`{leak-color}`, card "missed"). Step controls label "Unmask them".
  - Tricks in order: Plain number · Tamil digits · Devanagari digits · Full-width digits · URL percent-encoding · JSON scapes · HTML entities · Dots between groups · Space between every digit · Zero-width space inside · Split into three fields · Whole line in base64.
- **Button:** "↻ Another customer".
- **Under the hood:** normaliser maps back to original characters so the token keeps the disguise; E10 8.3% → 100%; E12 typo repair 0% → 99.2%.

5.9 Chapter 8: Tracing a leak

- **Headline:** "A leaked file names its source". **Tagline:** "Each system received a different token for the same customer. Leak one and watch Aran trace it back."
- **Data:** `POST /api/tour/tokens` → `customer`, `name`, `aadhaar`, `card`, `aadhaar_valid`, `card_valid`, `sinks[4]` {sink, purpose, aadhaar_token, card_token, after, aadhaar_token_passes, card_token_passes}. Leaking calls `POST /api/trace {dump: that sink's after}` → see page 5.
- **Scene: trace map** (SVG ~860 × 340):
  - Left: customer node (name + real Aadhaar, mono).
  - Middle: 4 sink nodes stacked (Log platform, CRM notes, Loan vendor, Marketing), each showing its own Aadhaar token in `{token-color}`; soft lanes from the customer to each.
  - Right: "Dark web · leaked file" node (`{leak-color}` when active).
- **Buttons:** "Leak Log platform" · "Leak CRM notes" · "Leak Loan vendor" · "Leak Marketing".
- **Animation on leak (about 4.2 s):** the chosen sink turns "leaked" (`{leak-color}` outline); a document icon flies from it to the dark-web node (1.1 s); after 1.7 s a **beam** (animated dashed curve, `{accent-color}`) draws from the dark-web node back to the sink Aran named; at 2.9 s that sink turns "found" (`{success-color}`) and the verdict appears: "✓ From the leaked text alone, Aran named **{top.sink}** ({matched} matching token(s))", or bad verdict "Aran named {sink or no system}".
- **Button:** "↻ Another customer".
- **Under the hood:** Theorem 2, E3 100% across 40 systems, link to the Leak trace page.

5.10 Chapter 9: Erasure

- **Headline:** "Erase everywhere, with proof". **Tagline:** "Each customer's tokens are locked by their own key per purpose. Destroy the key and every copy, anywhere, becomes meaningless."
- **Stage strip:** "1. Enrol" · "2. Send a loan" · "3. Erase Loans" (done / now / not yet).
- **Vault scene:**
  - Customer box: name, ref and masked Aadhaar (mono), or "No customer yet".
  - Four key boxes: KYC (locked, 🔒, "locked 5 yrs", `{warning-color}`), Loans, Service, Marketing. Ghost outlines before enrolment. After erasure the Loans key **shatters** (5 shards fly apart and fade, `{motion-slow}`) and shows "destroyed" in `{leak-color}`. Trying to erase KYC makes the KYC box **shake**.
  - Loan vendor's database box: "empty", then the token (mono), then struck-through/dimmed after erasure. Line "Bank reads it: {value}" (`{success-color}`, mono) → "unreadable, key destroyed" (`{leak-color}`).

- **Buttons:** “1. Enrol a customer” / “Enrol another” · “2. Send their loan application” (enabled after 1) · “3. Erase Loans” (danger, enabled after 2) · “Try erasing KYC” (secondary, enabled after 1).
- **Data flow:** 1 `POST /api/customers {}` → {ref}; then `GET /api/customers/{ref}`. 2 `POST /api/tour/customer-loan {customer_ref}` → event; take the Aadhaar finding’s token; `POST /api/detokenise {token, caller: "loan-vendor"}` → {status “ok”, value}. 3 `POST /api/erase {customer_ref, purpose: "Loans"}` → receipt; `POST /api/detokenise` again → status “erased”; `POST /api/receipts/verify` (receipt) and again with `tokens_unreadable + 1` (tampered). KYC: `POST /api/erase {purpose: "KYC"}` → 423 with “KYC records are under a retention lock until …”; show “🔒 Refused: {message}”.
- **Receipt panel** after erasure: header “Signed erasure receipt” + “entry {leaf_index+1} of {tree_size}”; line “{tokens_unreadable} token(s) made unreadable everywhere.”; **Merkle chain** (C27) recomputed in the browser with WebCrypto: “Your browser re-checks the receipt itself: {n} hashes from this erasure up to the log’s signed root.”; result chips “✓ in the log (checked in your browser)” and “✓ Ed25519 + ML-DSA-65 signatures valid”; tamper chip “✓ an edited copy of the receipt fails the check”.
- **Under the hood:** AES-256-GCM per (customer, purpose), RFC 6962 Merkle log, Ed25519 + ML-DSA-65, PMLA 5-year lock, legal confirmation still needed.

5.11 Chapter 10: Immune system

- **Headline:** “An immune system for data flows”. **Tagline:** “It learns what normal traffic looks like, then flags anything outside it, with no list of known attacks.”
- **Data:** `GET /api/tour/immune-radar` (poll 2 s) → `mode` (idle / learning / watching), `names[7]`, `self_lo` 0.1, `self_hi` 0.7, `windows[]` {at, src, sink, n, score, alert, z[7]}, `state`, `learn_left`, `next_window_in`, `window_seconds`. When watching also `GET /api/immune` → `alerts` (top 3).
- **Three modes:**
  - **Idle:** pulsing ring with “?”, “**Not trained yet.** It must first watch 6 minutes of normal traffic (real production logs replayed through the firewall).” Error in `{leak-color}` if `state.error`. Button “Learn normal traffic” → `POST /api/immune/learn {seconds: 360}`.
  - **Learning:** circular progress ring filling over 360 s with a mm:ss countdown from `learn_left`; “**Learning what ”normal” looks like…**” and “Real log lines are flowing through the firewall right now. You can explore other chapters and come back.”
  - **Watching: Radar** (C29) with the learned band between 0.1 and 0.7 in `{band-color}`, grid rings at 0.25/0.5/0.75/1, axis labels, the last 8 windows as polygons (normal soft, alert in `{leak-color}`, newest emphasised). Side panel: legend (normal range it learned · a 30-second traffic window · outside normal: alert), “Next check in **{next_window_in}s** · {detectors} detectors”, drill buttons “⚡ Send data to an unknown site” · “⚡ Bulk export” · “⚡ Insider reads many numbers” · “⚡ Switch the firewall off” (`POST /api/drills/{kind}`), then “Drill”{kind}” sent. Watch the radar at the next check.” Below: up to 3 alert boxes that **pop in** (“⚠ {src} → {sink}” + reasons).
- **Link:** “Open the full immune console →”.
- **Under the hood:** V-detector negative selection over 7 features per (source, sink, 30 s window), memory on confirm, E8 100% at 0.6% false alarms.

5.12 Chapter 11: AI chatbots

- **Headline:** “Safe AI chatbots”. **Tagline:** “Staff can ask an AI about a customer. The AI only ever sees placeholders, and each role sees only what it is allowed to.”
- **Role chips:** Customer care (default) · KYC officer · Analyst.

- **Prompt textarea:** pre-filled from `GET /api/tour/generate`: “Customer Meena Iyer, mobile {random}, says card {card} was charged twice. Her Aadhaar is {aadhaar}. Draft a polite reply.”
- **Buttons:** “Ask the AI through Aran” · “Compare all three roles” (both disabled when empty).
- **Data:** `POST /api/tour/gateway {role, text}` → `reply`, `provider_saw[0].content`, `provider_reply`, `findings_in_prompt`, `findings_in_reply`, `ms` {tokenise, provider}, `provider` (name, e.g. “echo stand-in (no external AI call)”), `role`, `audit_ok`, `audit_size`, `audit_last` {shown {type: count}, masked}.
- **Scene: four-step chat flow** (900 ms per step, auto-plays):
  1. Staff bubble “Staff (Customer care)” with the original text.
  2. Shield strip “ARAN · {findings_in_prompt} values replaced · {ms.tokenise} ms” + provider bubble “What the AI provider saw” with placeholders like `<AADHAAR_1>` highlighted in `{token-color}`.
  3. Provider bubble “AI reply ({provider})” with placeholders highlighted.
  4. Staff bubble “What the staff member sees” with `reply`, and small line “shown Mobile 1, … · masked {n} · audit chain ✓ intact” (or “✗ broken” in `{leak-color}`).
- **Compare:** three cards side by side (stacked on phone): “{Role} sees” + that role’s reply.
- **Under the hood:** same value same placeholder, reply re-scanned, role-based resolution, hash-chained audit log, `ARAN_LLM_URL` for a real model.

5.13 Chapter 12: Evidence

- **Headline:** “The evidence”. **Tagline:** “Read live from the result files the experiments wrote. Anyone can re-run them.”
- **Data:** `GET /api/results` (once) → `"E1-E4 core".data` (e1_total, e2, e3, e4), `"E5-E9 AI and immune".data` (e8_immune, e9_firewall).
- **Three stat tiles with count-up:** “{e2.token_flagged} / {e2.injected} · tokens mistaken for real” · “{e3.dump_1_accuracy×100}% · leaks traced to the right one of {e3.sinks} systems” · “{e8_immune.ais.tpr×100}% · data-flow attacks caught”.
- **Two versus bars** (each row: label, bar, count-up number): “OTP false alarms on real logs · fewer is better”: Simple rules `e1_total.otp_regex` (`{leak-color}` bar) vs Aran `e1_total.otp_context` (`{success-color}` bar). “Sensitive values removed (%) · more is better”: `e9_firewall.prototype_regex_recall` vs `e9_firewall.leak_recall`. Bars grow from 0.
- **Button:** “All results and live test runs →” (Results page).
- **“Not finished yet”** grid of 4: Names and addresses · 74% and 90% caught; Speed · ~300 lines/s per core; Rust core planned; Erasure in law · needs a lawyer’s confirmation; Keys · in memory here; an HSM in a bank.
- **Regulation chips:** DPDP Act 2023 and Rules 2025 · CERT-In 2022 · UIDAI Data Vault · RBI tokenisation · RBI FREE-AI · PCI DSS · PMLA retention.

5.14 Chapter 13: Glossary

- **Headline:** “Glossary and team”.
- **Glossary grid** (term bold + one line): Token · Check digit · Verhoeff / Luhn · Sink · False alarm · Calibrated probability · Normaliser · Crypto-shredding · Merkle log · Immune layer · DPDP Act (definitions as in the current site).
- **“Built at KARE”** team: IS Inbathamizhan S · Team lead · Cybersecurity; AA Aasif Ahmed A · Cybersecurity; SA SN Arun Vijay · AI/ML; BM Bharadhwaj M · AI/ML.
- **Footer line:** “Banking, Financial Services and FinTech · Prevent Sensitive Data Leakage · github.com/itzz-inbathamizhans/aran”.

- Bottom nav right button: “Open the full console →”.

---

6. Page 2: Overview

Route: `#overview`. **Eyebrow:** “DATA-FLOW FIREWALL FOR INDIAN BANKS”. **Title:** “Real numbers pass the checksum. Aran’s tokens never do.” **Lede:** “This console runs Aran’s actual code. Send a request, watch it get protected, trace a leak, erase a customer, and see every number update from the live server.”

Data: `GET /api/stats` (poll 3 s), `GET /api/events?limit=6` (poll 3 s), `GET /api/immune` (poll 4 s).

Section	Content and binding	States
Hero pair	“Latest Aadhaar this server protected”: ID strip (real) of the raw Aadhaar from the newest event that has a 12-digit Aadhaar token (`before.slice(start,end)`), arrow “↓ tokenised before it left the API”, ID strip (token) of `token`	Empty: card “No traffic yet on this server. Send a first request to see Aran work.” + button “Send a request” (→ Live traffic)
Stat row 1 (4 tiles)	Requests protected `requests` · note “{customers} enrolled customers”; Values tokenised `values_protected` · “{values_passed_raw} passed raw”; Added latency p95 “{p95_ms} ms” · “median {p50_ms} ms” (or “–” / “no requests yet”); Immune alerts `alerts` · “{open_alerts} open · mode: {immune.mode}” (danger tone when `open_alerts` > 0)	Values count smoothly when they change
Stat row 2 (4 tiles)	Tokens issued `tokens_issued` · “{vault_tokens} purpose-bound”; Keys destroyed `keys_destroyed` · “Merkle log: {merkle_size} entries”; Normal traffic replay “On”/“Off” from `replay.running` · “{replay.lines_sent} real log lines sent”; Server uptime “{uptime_s/60} min” · “models loaded in {model_load_seconds}s”	
Values found by type (card, left)	Table: type label · count from `by_type`, sorted high to low. Suggested redesign: horizontal bar per type in the type’s tint	Empty: “Nothing yet.”
Latest requests (card, right)	Table of 6: “{source} → {sink}” · “{findings.length} found” · “{n}s ago”; button “Open live traffic →”	Empty: “Nothing yet.”
Latest immune alerts (card, only if any)	Up to 3 rows: `{leak-color}` dot + “{src} → {sink}: {reasons[0]}”; button “Open immune layer →”	Hidden when no alerts

Numbers use Indian grouping (1,00,000). Times are relative (“12s ago”, “3m ago”, “1h ago”).

---

7. Page 3: Live traffic

Route: `#traffic`. **Eyebrow:** “LIVE TRAFFIC”. **Title:** “Send a request through Aran”. **Lede:** “Pick a bank request or paste your own text. Aran detects every sensitive value, replaces it before it reaches the sink, and shows its reasoning.”

Data: `GET /api/config` (once), `GET /api/events?limit=15` (poll 4 s), `POST /api/sample`, `POST /api/process`.

7.1 Composer card

- **Tabs:** “Bank requests” · “Your own text”.
- **Bank requests tab:**
  - Segmented radio: “Enrolled customer (registry knows them)” (default) · “Stranger (not in the registry)” → `who`.
  - Button grid (3 × 2 desktop, 1 column phone), one secondary button per sample: label (e.g. “Loan application API”) + small line “{source} → {sink}” (e.g. “loan-origination → loan-vendor”). Click → `POST /api/sample {kind, who}`. All disabled while busy.
- **Your own text tab:**
  - Mono textarea (5 rows), placeholder “Paste a log line, JSON body or chat prompt. Use generated test numbers only, never a real person’s.” (max 20,000 chars; consider a live counter).
  - Row: “Source” text input (default `my-app`) · “Sink” select listing “{name} ({purpose})” (default log-platform) · “Protect” primary (disabled while empty or busy) → `POST /api/process {text, source, sink}`.
- Error note under the card (400 “sink names are lower-case letters, digits, dots and dashes”).

7.2 Result (latest or selected event)

- **Meta row:** “**{source} → {sink}**” + “monitor-only” danger pill when `bypass`; right: mono “{n} found · {latency_ms} ms · {ago}”.
- **Two cards side by side:** “Before (what the app sent)” with Marked text (clickable marks) + hint “Click a highlighted value to see why it was caught.” · “After (what {sink} stored)” with Token text. Phone: stacked.
- **Detail card** (when a finding is picked): “Why the detector flagged this {Type}”; row “Calibrated probability **{p}%**”; Confidence bar with the sink’s threshold tick; note “Marker = this sink’s threshold ({threshold}). At or above it the value is replaced.”; evidence table (label · value, mono); extra rows “Own-customer / OTP registry: hit” and “Typo repaired to a registered customer: yes” when set; result line “replaced by `{token}`” or “below threshold, passed raw”.
- **Findings table:** Type · Value (mono, raw) · Became (token mono, or “passed raw” danger pill) · Confidence · Registry. Rows clickable → open the detail card. Empty: “Nothing sensitive found.”
- **Animation:** when a new result arrives, Before marks appear, then After tokens fade in (`{motion-medium}`); the picked mark gets an outline; detail card slides open.

7.3 Recent traffic card

Table (poll 4 s, 15 rows): When (“12s ago”) · Flow (“source → sink”) · Found · Latency. Rows clickable to load that event above. Default: newest event is shown on first load.

States: page renders nothing until config loads (show a skeleton); busy disables sample buttons; empty recent table shows “No traffic yet.”

---

8. Page 4: Customers and erasure

Route: `#customers`. **Eyebrow:** “PURPOSE-BOUND CRYPTO-SHREDDING (M5)”. **Title:** “Destroy one key. Every copy is erased, with a signed receipt.” **Lede:** “Each customer has one key per purpose. Tokens sent to sinks are encrypted under that key. Destroy the key and every token for that purpose, in every log and vendor copy, can never be read again.”

Data: `GET /api/customers` (once, reload after changes) → list of {ref, name, masked {AADHAAR, CARD, MOBILE, PAN…}, keys[] {purpose, alive, tokens, locked_until, receipt}}. `GET /api/customers/{ref}` → same + `tokens[]` {token, sink, purpose, type, at} (last 50). `POST /api/customers {}` · `POST /api/erase` · `POST /api/detokenise` · `POST /api/receipts/verify` · `GET /api/merkle` → {size, root, recent[] {event, customer, purpose, at}}.

Layout: two columns, list narrow left, detail wide right; phone: list first, detail below (or list as a drawer).

8.1 Customer list (left)

- Header “Customers” + ghost button “+ Add” → `POST /api/customers` then opens the new one.
- Note: “Generated identities. Identifiers are checksum-valid but belong to no one.”
- List items: **ref** (mono, e.g. C0007) + name; right small text “{n} erased” (if any key destroyed) or “{total tokens} tokens”. Selected item highlighted. Consider a search box `[client-side filter]`.
- Default about 25 customers.

8.2 Detail (right)

- **Empty:** card “Pick a customer to see their keys and erase a purpose. Send some bank requests first so they have tokens in the sinks.”
- **Customer card:** title “{ref} · {name}”; chips of masked identifiers (mono: “AADHAAR XXXX XXXX 1234”, “CARD XXXX XXXX XXXX 5678”, “MOBILE XXXXXX4321”, “PAN ABXXXXXX4F”); **keys table**: Purpose · Key (“alive” ok pill / “destroyed” danger pill) · Tokens (mono) · action:
  - alive + locked: small `{warning-color}` text “locked until {date}” with tooltip “PMLA: KYC records are kept 5 years”.
  - alive + not locked: danger button “Erase” → **confirm dialog** (C31): “Destroy {ref}’s {purpose} key? Every {purpose} token for this customer becomes permanently unreadable.” → `POST /api/erase`.
  - destroyed: ghost button “Receipt” (shows the stored receipt).
  - Errors: 423 retention lock, 404 no such customer.

    - Animation on erase: the row's pill flips alive → destroyed, a small key-shatter icon.
- **Receipt card** (C26), after erasing or clicking Receipt: "Erasure receipt"; table: Customer · Purpose · Tokens made unreadable · Signed at (local date-time) · Merkle leaf / tree size · Merkle root (mono, truncated with full value on hover/copy) · Signature (Ed25519) · Post-quantum signature (ML-DSA-65). Buttons: "Verify receipt" → ok note "✓ Both signatures (Ed25519 and post-quantum ML-DSA-65) valid and the event is included in the signed Merkle root." / err "✗ Receipt does not verify."; "Try tampering with it" (adds 1 to tokens_unreadable) → ok note "✓ Changing one number (tokens made unreadable) broke it: signature invalid, inclusion invalid."; "Copy JSON" (copies the receipt; show a brief "Copied" toast). Suggested: add the in-browser Merkle chain (C27) here too.
- **Read a token back card:** "Try to read a token back" + note "Detokenisation is logged per caller. After erasure it fails for good."; chips of the last 6 tokens (mono token + sink), click to try; mono input "token" + "Detokenise" (secondary, disabled when empty). Result: ok "Value: `{value}`" or err with `reason` (erased: "…key destroyed…"; unknown: "Not a vault token. Vaultless FF1 tokens are reversed only inside the bank's key boundary.").

8.3 Transparency log (full width, below)

- "Transparency log", note "{size} entries · root {first 32 hex}…".
- Table of the 10 most recent: Event (KEY_DESTROYED in `{leak-color}`, others normal) · Customer (mono) · Purpose · When. Suggested: new entries slide in at the top.

---

9. Page 5: Leak trace

Route: `#trace`. **Eyebrow:** "LEAK ATTRIBUTION (M3)". **Title:** "A leaked token names the system that leaked it." **Lede:** "Every sink receives different tokens for the same customer. Paste a leaked file and Aran tells you which sink it came from, with the evidence."

Data: `GET /api/config` (sinks), `GET /api/leaked-dump?sink={name}&lines=20` → {sink, dump}, `POST /api/trace {dump}` → `candidates`, `matched`, `ranking[]` {sink, tokens, share}, `top` (first of ranking or null), `items[]` {value, sink, how ("vault" / "issued-token index" / "FF1 offset + registry" / null)} (up to 200), `raw_identifiers_in_dump`.

9.1 Input card

- Row: label "Take what this sink really stored" + sink select · secondary "Load its last 20 lines" → fills the textarea. Empty dump → error note "{sink} has not received anything yet. Send some traffic to it first."
- Mono textarea (8 rows, max 200,000 chars), placeholder "Paste a leaked dump, or load one from a sink above. You can mix lines from several sinks."
- Primary "Trace the leak" (disabled when empty or busy).
- Suggested: drag-and-drop a .txt or .log file onto the textarea `[client-side only]`.

9.2 Result (two cards)

- **Verdict card:** title "Most likely source: **{top.sink}**" (`{accent-color}`) or "No Aran tokens found". Line "{matched} of {candidates} candidate values matched a token issued by this server." When `raw_identifiers_in_dump` > 0, add in `{leak-color}` bold: "{n} raw identifiers in the dump were never tokenised, so they leaked before Aran or through a bypass." Then

rank bar rows (C17) per sink: name · bar (share) · “{tokens} · {share}%”. Bars grow from 0 in order (`{motion-medium}`, 80 ms stagger).

- **Evidence card:** “Evidence per value” table (first 60): Value (mono) · Sink (“no match” muted) · How.
- **States:** before trace nothing below the input; busy label “Tracing…”; error note.

---

10. Page 6: Immune layer

Route: `#immune`. **Eyebrow:** “ARTIFICIAL IMMUNE SYSTEM”. **Title:** “Learns what normal looks like, then flags what is not.” **Lede:** “Every {window_seconds} seconds the firewall closes one flow record per source and sink: how many messages carried sensitive values, how many left raw, which data classes, how often tokens were read back. The immune layer learns”self” from this server’s own traffic, then scores every new record with negative selection.”

Data: `GET /api/config` (drills), `GET /api/immune` (poll 2 s) → `state` {mode, learned_records, self_radius, detectors, val_fpr, learn_until, window_seconds, error, pairs[]}, `alerts[]` (up to 100) {id, at, src, sink, score, n, raw_frac, detok_rate, classes[], features {}, reasons[], label (null / “attack” / “normal”)}, `open_windows`, `closed_windows`. `GET /api/stats` (poll 3 s) → `replay` {running, lines_per_second_per_flow, lines_sent, flows[], log_systems}.

10.1 Stat row (4 tiles)

Mode `state.mode` (idle / learning / watching; tint: idle neutral, learning `{warning-color}`, watching `{success-color}`) · note “{seconds}s left” / “learned from {learned_records} records” / “not trained yet”. Detectors `detectors` · “self radius {self_radius}”. Held-out false alarms `val_fpr` as % · “on normal windows it did not train on”. Flow windows `closed_windows` · “{open_windows} open now”.

Suggested: embed the radar from tour chapter 10 (`GET /api/tour/immune-radar`) at the top of this page in watching mode.

10.2 Step cards (numbered, left to right)

1. **Normal traffic:** text “Replays real production log lines (Loghub, {log_systems} systems) through the firewall as the bank’s everyday traffic. About one line in ten carries a bank event for an enrolled customer.” Number input “Lines per second per flow” (0.2–5, step 0.2, default 1). Button “Start replay” (`POST /api/replay/start {rate}`) or “Stop replay” (`POST /api/replay/stop`). Status line “Running/Stopped · {lines_sent} lines sent · flows: core-banking -> log-platform, …”. Suggested: a small live pulse dot while running.
2. **Learn self:** text “Starts the replay if needed, collects flow records for the chosen time, then fits the detectors with the self radius tuned on held-out windows.” Number input “Seconds” (30–1800, default 360). Button “Learn normal traffic” / “Re-learn” (disabled while learning) → `POST /api/immune/learn {seconds}`. Error `state.error` (e.g. “only 4 flow windows seen; keep normal traffic running and learn again”). “Self covers: {pairs}”. Suggested: progress ring with countdown while learning.
3. **Run a drill** (full width): text “Each drill performs a real action on the live firewall. The immune layer is not told about it; it has to notice the flow window on its own when the window closes (up to {window_seconds}s).” Grid of 4 drill buttons (labels from config), disabled unless watching, with hint “Available once the immune layer has learned normal

traffic.” On success, ok note “{done message} Watch the alerts below.” (e.g. “Exported 250 loan applications to the loan vendor in one window.”). Error 409 “Teach the immune layer normal traffic first”. Suggested: a countdown to the next window close.

10.3 Alerts list

- Empty: “No alerts.”
- Each alert (C18): “**{src} → {sink}**” · right “{ago} · score {score}”; bullet list of `reasons`; mono line “n={n} raw={raw_frac} detok={detok_rate} classes={classes}”; actions: danger “Confirm attack” → `POST /api/immune/alerts/{id}/label {label: "attack"}`; ghost “Mark as normal” → `{label: "normal"}`. After labelling: pill “confirmed attack (memory cells added)” (danger) or “marked normal (detectors removed)” (ok).
- New alerts slide in at the top with a short pulse in `{leak-color}`. Suggested: filter chips All · Open · Confirmed · Normal `[client-side]`.

---

11. Page 7: Results and tests

Route: `#results`. **Eyebrow:** “EVIDENCE”. **Title:** “Results from the experiment scripts, not from slides”. **Lede:** “These tables are read from the result files the experiment scripts wrote. Run the test suite or re-run E1 to E4 on this server and watch the output.”

Data: `GET /api/results` (once) → map of `"E1-E4 core"`, `"E5-E9 AI and immune"`, `"E10-E12 normaliser and registries"`, each {file, modified, data}. Jobs: `POST /api/jobs/{name}` starts (`tests` or `e1-e4`), `GET /api/jobs/{name}` (poll 2 s while running; 404 means never run) → {name, cmd, started, finished, returncode, output}.

11.1 Job cards (two side by side)

- **Run the test suite:** note “pytest: NIST FF1 vectors, Verhoeff and Luhn, token invariants, normaliser, registries, crypto-shredding and Merkle receipts.”
- **Re-run experiments E1 to E4:** note “False alarms on 32,000 real Loghub lines, token separability, leak attribution, mask leakage. Takes a few minutes; the table below updates when it finishes.”
- Button: “Run now” / “Running…” (disabled) / “Run again”. Status: “Started {n}s ago” (running, with spinner) · “✓ Passed in {n}s” (`{success-color}`) · “✗ Exit code {n}” (`{leak-color}`). Output: scrollable mono block (last 4,000 chars), auto-scrolls to the bottom while running.

11.2 Result tables (cards)

- **E1 to E3 on {e1_total.lines} real production log lines** with source note “({file}, written {date})”. Table: Test · Plain pattern matching · Aran. Rows: OTP false alarms (`otp_regex` vs `otp_context`) · Card false alarms (Luhn + issuer prefix) (`card_regex` vs `card_luhn_iin`) · Aadhaar false alarms (Verhoeff) (`aadhaar_regex` vs `aadhaar_verhoeff`) · Tokens wrongly flagged as real (of `e2.injected`) (“not separable” vs `e2.token_flagged`) · Leaked dumps traced to the right sink ({e3.sinks} sinks, 1 token) (“access logs only” vs `e3.dump_1_accuracy`%).
- **E4: what common masks still leave:** Mask · Candidates left · Bits left (from `e4[]` {mask, candidates, bits_left}). Suggested: a horizontal bar of bits left per mask.

- **E9: the wired firewall on {e9_firewall.lines} real lines of unseen systems:** Sensitive values removed (`prototype_regex_recall` vs `leak_recall`) · Clean real lines wrongly changed (`prototype_clean_lines_flagged` vs `clean_real_lines_changed`) · Tokens re-detected as real IDs (– vs `tokens_redetected_as_real_ids`).
- **E8: immune layer on six exfiltration scenarios:** Method · Attacks caught · False alarms: Immune layer (`ais.tpr`, `ais.fpr`), Static rules, Isolation Forest.
- **E10: hidden identifiers removed, by evasion trick:** one row per trick, one column per condition (from `e10_evasion.conditions[*].by_trick[trick].rate`). Suggested: a heat-map of the cells using `{leak-color}` → `{success-color}` with the % printed in each.
- A section is hidden if its result file is missing. Suggested empty state: “No result file yet. Run the experiment to create it.”

Note: E13 to E19 (ledger, AI gateway, red team, speed, post-quantum) are not served by `/api/results` yet `[needs endpoint]`; the README has their tables.

---

12. Page 8: How it works

Route: `#about`. **Eyebrow:** “HOW IT WORKS”. **Title:** “Five mechanisms, each built on a named theory.” No API calls.

- **Mechanism cards** (grid, 5 cards; 3 + 2 desktop, 1 column phone): eyebrow code, title, one sentence, theory chips.
  - M1 Checksum-complement tokens · “Tokens keep the 12 or 16 digit shape but always fail Verhoeff or Luhn, so any scanner can tell a token from a real number.” · Dihedral group D5 · FF1 (NIST SP 800-38G)
  - M2 Evidence-fusion detector · “A calibrated probability per value from the checksum, the context, NER, the own-customer registry and the trace-bound OTP registry.” · Bayes’ rule · Isotonic calibration · Cuckoo filter
  - M3 Per-sink tokens · “Every destination gets different tokens, so a leaked token names the system that leaked it.” · PRP families · Maximum likelihood
  - M4 Leakage ledger · “Caps how many bits any group of destinations can learn about one customer.” · Min-entropy (update the copy: the ledger is now built in `research/firewall`, E13)
  - M5 Purpose-bound crypto-shredding · “One key per customer per purpose; destroying it erases every copy everywhere, with a signed Merkle receipt.” · AES-256-GCM · Ed25519 + ML-DSA-65 · RFC 6962 Merkle log
- **“What runs behind this console”** card, bullets: models trained by `app/server/build_models.py` on public corpora; every request goes through `research/firewall` (normaliser, registries, typo repair, then per-sink or vault tokens); keys in memory only, forgotten on restart; customers are generated; honest note on DPDP erasure needing legal confirmation.
- **Team card:** “Team · Kalasalingam Academy of Research and Education” with the four member cards (C32).
- Suggested additions: a simple architecture diagram (sources → normaliser → detector → registries → token/vault → sinks, with the immune layer watching flows and the AI gateway as one sink) and an “Add AI layers and post-quantum” card (AI gateway; post-quantum receipts and key sealing).

---

13. API reference (every endpoint the UI uses)

Base path `/api`. JSON in and out. Interactive docs at `/docs` while the server runs. Error replies carry `detail` (show it as the error note).

Method	Path	Body / query	Returns (main fields)	Used on
GET	`/health`		ready, error	Shell (poll 2 s until ready)
GET	`/config`		sinks[] {name, purpose, threshold}, purposes, samples, sample_sinks, sample_sources, public_key, window_seconds, drills	Traffic, Trace, Immune
GET	`/stats`		requests, values_protected, values_passed_raw, by_type, p50_ms, p95_ms, alerts, open_alerts, customers, tokens_issued, vault_tokens, keys_destroyed, merkle_size, uptime_s, immune, replay, model_load_seconds	Overview, Immune
POST	`/process`	text (≤20,000), source, sink, trace_id?	event (4.1)	Traffic, Tour 1
POST	`/sample`	kind, who (customer / stranger), sink?	event	Traffic, Tour 6
GET	`/events`	limit (1–300, default 50)	event[] newest first	Overview, Traffic
GET	`/customers`		[{ref, name, masked, keys[]}]	Customers
POST	`/customers`	name?	customer	Customers, Tour 9
GET	`/customers/{ref}`		customer + tokens[] (404 if unknown)	Customers, Tour 9
POST	`/detokenise`	token, caller	token, value, status (ok / erased / unknown), reason	Customers, Tour 9

Method	Path	Body / query	Returns (main fields)	Used on
POST	`/erase`	customer_ref, purpose	receipt {event {customer, purpose, tokens_unreadable, at, …}, leaf_index, tree_size, root, proof[], signature, public_key, pq_signature, pq_public_key, alg}; 423 retention lock, 404 unknown	Customers, Tour 9
POST	`/receipts/verify`	receipt	valid, signature_valid, included_in_log, post_quantum_valid, alg	Customers, Tour 9
GET	`/merkle`		size, root, recent[]	Customers
POST	`/trace`	dump (≤200,000)	candidates, matched, ranking[], top, items[], raw_identifiers_in_dump	Trace, Tour 8
GET	`/leaked-dump`	sink, lines (1–200)	sink, dump	Trace
GET	`/immune`		state, alerts[], open_windows, closed_windows	Immune, Overview, Tour 10
POST	`/immune/learn`	seconds (30–1800)	state	Immune, Tour 10
POST	`/immune/alerts/{id}/label`	label (attack / normal)	alert	Immune
POST	`/drills/{kind}`		done (message), window_seconds; 409 if not watching	Immune, Tour 10
POST	`/replay/start`	rate (0.2–5)	replay status	Immune
POST	`/replay/stop`		replay status	Immune
GET	`/results`		result files by name	Results, Tour 12
POST	`/jobs/{name}`	name = tests / e1-e4	job	Results
GET	`/jobs/{name}`		job (404 if never run)	Results
POST	`/tour/checksum`	digits (12 or 16)	kind, digits, valid, steps[], final/total, correct_check_digit	Tour 4
GET	`/tour/generate`		aadhaar, card	Tour 4, Tour 11
POST	`/tour/tokens`		customer, name, aadhaar, card, sinks[4]	Tour 8
POST	`/tour/false-alarms`		lines, systems, naive, naive_total, aran, aran_total, cells, examples, seconds	Tour 3

Method	Path	Body / query	Returns (main fields)	Used on
POST	`/tour/customer-loan`	customer_ref	event	Tour 9
POST	`/tour/evasion`		customer, aadhaar, rows[12], caught	Tour 7
POST	`/tour/packet`	kind?	event + kind	Tour 1
POST	`/tour/build-token`	sink, aadhaar?	aadhaar, body, rounds[10], right_check, offset, token, firewall_token, matches…	Tour 5
GET	`/tour/immune-radar`		mode, names, windows[], state, learn_left, next_window_in, window_seconds	Tour 10
POST	`/tour/gateway`	role, text (≤4,000)	reply, provider_saw, provider_reply, findings_in_prompt, ms, provider, audit_ok, audit_last	Tour 11

All state lives in server memory and resets on restart; design copy should never promise that data is saved.

---

14. Responsive and accessibility rules

Width	Shell	Content
Phone < 720 px	Sidebar becomes a drawer behind ☰; top bar text shortens	One column; two-column sections stack; tables scroll sideways inside their card; tour dots become a dropdown; SVG scenes scroll inside their frame; button grids become one column
Tablet 720–1100 px	Sidebar narrow	Two columns where each column ≥ 320 px
Desktop > 1100 px	Full sidebar	Content max width about 1200 px

- No sideways scroll of the whole page.
- Touch targets at least 44 × 44 px.
- Visible keyboard focus everywhere (`{focus-ring-color}`); tour arrows work with keys.
- Each SVG scene has a one-line description (given per scene above).
- Real / token / leak always paired with ✓ ⊘ ! or words.
- `prefers-reduced-motion`: skip packet flights, shatters, shakes and count-ups; jump step-by-step scenes to their final frame; keep fades under 150 ms.
- Identifiers and numbers in `{font-mono}`; Tamil text in `{font-tamil}`.
- Error text is announced (live region) and says what to do next.

---

15. Design checklist before handoff

- [ ] Token table filled for light and dark; every text pair ≥ 4.5:1.
- [ ] Component board (C1 to C33) with every state.
- [ ] Shell: starting, load failed, normal; phone drawer open and closed.
- [ ] All 13 tour chapters, each with loading, success and error; step scenes at first, middle and last frame.
- [ ] Overview: empty server and busy server.
- [ ] Live traffic: both tabs, with a finding picked, with “passed raw” and “monitor-only”.
- [ ] Customers: no selection, customer selected, after erasure with receipt, KYC refused, detokenise ok / erased / unknown.
- [ ] Leak trace: empty, loaded dump, result with a winner, “No Aran tokens found”, raw identifiers warning.
- [ ] Immune: idle, learning, watching, drill sent, alert open, alert labelled.
- [ ] Results: job never run, running, passed, failed; missing result file.
- [ ] Every number in the design has a binding from section 13, or is marked `[needs endpoint]`.
- [ ] Phone and desktop frames for every page.
- [ ] Reduced-motion version noted for every animation.

---

16. Optional extra pages (need new endpoints)

Only if there is time; each needs server work first, so mark them `[needs endpoint]` in the design.

Page	What it would show	What the server needs
Leakage ledger	Bits each vendor group has learned per customer, with the cap	Expose the M4 ledger from `research/firewall`
AI gateway console	Full chat with role switch and the hash-chained audit log	`/api/gateway/audit` listing entries
Post-quantum	CBOM table and Mosca priority table	Serve `research/results_pqc.json` and `python -m pqc.cbom` output
Settings	Sink thresholds, customer count, LLM URL	Write endpoints for config (none today)
Results E13 to E19	Ledger, gateway, red team, speed, post-quantum tables	Add those result files to `RESULT_FILES` in `engine.py`